MBLP131 · HAFTA 2

İşletim Sistemleri Temelleri

Hafta 2 · Tanımlar, Hedefler, Bileşenler ve İS Yapıları

İşletim sistemi tam olarak nedir, ne işe yarar ve nasıl inşa edilir?

Elektronik Haberleşme Teknolojisi Programı · Meslek Yüksekokulu · İstanbul Okan Üniversitesi

GEÇEN HAFTA

Dört Fikirde 1. Hafta

Bilgisayar komutları izler

Girdi → işlem → çıktı; program, bellekte verinin hemen yanında durur.

İlk işletim sistemi başlar

…ve diğer her şeyi yönetir; böylece öteki programlar çalışabilir.

Boşta duran CPU pahalıdır

Toplu işleme, çoklu programlama ve zaman paylaşımı, işlemciyi meşgul tutmak için icat edildi.

UNIX 1969, Linux 1991

Küçük, birleştirilebilir araçlar; Linux, 9. haftadan itibaren uygulamalı olarak kullanacağımız sistemdir.

Bugün bu gayriresmî tanımı kesin bir tanımla değiştiriyor ve işletim sistemlerinin gerçekte nasıl inşa edildiğine bakıyoruz.

Bugünün Gündemi

1. **Tanımlar** İşletim sistemi tam olarak nedir? İki klasik bakış açısı.
2. **Hedefler** Kolaylık, verimlilik, gelişebilirlik — ve aralarındaki ödünleşimler.
3. **Bileşenler** Donanım, işletim sistemi, uygulamalar, kullanıcılar; kesmeler, depolama, çift mod.
4. **Yapılar** Basit, monolitik, katmanlı, mikroçekirdek, modüler, hibrit.
5. **Uygulama** Gerçek C kodu ve strace ile sistem çağrılarını izlemek.

1. BÖLÜM

İşletim Sistemi Tam Olarak Nedir?

Gayriresmî bir betimlemeden kesin bir tanıma.

TANIMLAR

Karşılaşacağınız Üç Tanım

Aracı

Bilgisayarın kullanıcısı ile bilgisayar donanımı arasında aracılık eden bir program.

— Silberschatz, Galvin ve Gagne

Çekirdek

Bilgisayarda her zaman çalışan tek program. Geri kalan her şey ya bir sistem programı ya da bir uygulama programıdır.

— Silberschatz, Galvin ve Gagne

Yönetici

Programların yürütülmesini denetleyen ve bilgisayarın kaynaklarını (işlemci, bellek, dosyalar, aygıtlar) yöneten yazılım.

— Stallings (özetlenerek)

Çalışma tanımımız: işletim sistemi (İS), donanımı yöneten ve programlara üzerinde çalışabilecekleri daha basit, daha güvenli bir makine sunan yazılım katmanıdır. Bellekte sürekli bulunan özüne çekirdek (kernel) denir.

Genişletilmiş Makine ve Kaynak Yöneticisi

Andrew Tanenbaum işletim sistemini iki yönden tanımlar: yukarıdan aşağıya ve aşağıdan yukarıya.

Yukarıdan aşağıya: genişletilmiş makine

- Karmaşık donanımı temiz soyutlamaların arkasına gizler.
- Disk sektörlerini ve manyetik izleri değil, dosyaları görürsünüz.
- Kesme hatlarını ve yazmaçları değil, süreçleri görürsünüz.
- Amaç: bilgisayarı programlamayı ve kullanmayı kolaylaştırmak.

Aşağıdan yukarıya: kaynak yöneticisi

- Birçok program ve kullanıcı tek bir makine için yarışır.
- **Zamanda çoğullama:** sırayla kullanmak (CPU, yazıcı).
- **Alanda çoğullama:** parçalara bölerek paylaşmak (bellek, disk alanı).
- Amaç: kaynakları adil, verimli ve güvenli biçimde dağıtmak.

İki bakış da aynı yazılımı anlatır. Bu dersin 3–7. haftaları büyük ölçüde kaynak yöneticisi bakışıdır: süreçler, CPU, bellek, dosyalar.

TANIMLAR

Çekirdek, Sistem Programları ve Uygulamalar

İşletim sistemiyle birlikte gelen her şey "çekirdek" değildir.



Her katman yalnızca altındaki katmanın hizmetlerini kullanır.

Kim kimdir?

- **Çekirdek:** her zaman bellektedir, ayrıcalıklı bir modda çalışır, donanımla doğrudan konuşur.
- **Sistem programları:** işletim sistemiyle birlikte gelir ama sıradan programlar gibi çalışır; sistemi kullanılabilir kılar.
- **Uygulamalar:** asıl işinizi yapmak için yüklediğiniz her şey.
- Linux, tam anlamıyla yalnızca çekirdektir; bir "Linux dağıtımı" buna sistem programlarını (çoğunlukla GNU'dan) ekler.

TARTIŞMA

İşletim Sisteminin Bir Parçası mı?

Tek bir resmî yanıt yok; asıl mesele de bu. Önce oylayın, sonra tartışın.

Web tarayıcısı?

1998'de ABD hükümeti, kısmen Internet Explorer'ı Windows'la birlikte paketlediği için Microsoft'a dava açtı. Tarayıcı "Windows'un bir parçası" mı? Avukatlar bunu yıllarca tartıştı.

Grafik masaüstü?

Windows'ta sisteme sıkı sıkıya bağlıdır. Linux'ta ise çekirdeğin üzerinde çalışan, değiştirilebilir bir programdır (GNOME, KDE, ...).

Uygulama mağazası?

Telefonda çoğu kullanıcı uygulama mağazasını "sistemin kendisi" sayar. Teknik olarak özel izinlere sahip bir uygulamadır.

Kullanışlı bir kural: çekirdek kesinlikle işletim sistemidir. Çekirdeğin ötesinde "işletim sistemi", üreticinin standart sistem olarak sunduğu her şeydir.

2. BÖLÜM

İşletim Sistemi Ne İşe Yarar?

Üç tasarım hedefi ve bu hedeflerin neden farklı yönlere çektiği.

TASARIM HEDEFLERİ

Her İşletim Sisteminin Üç Hedefi

William Stallings, bir işletim sisteminin amaçlarını üç kelimeyle özetler.

1. Kolaylık

- Bilgisayarı kullanmayı ve programlamayı kolaylaştırır.
- Dosyalar, pencereler, kabuk, basit sistem çağrıları.
- Kullanıcıların donanımı bilmesi gerekmez.

2. Verimlilik

- Donanımı iyi kullanır: CPU meşgul, bellek faydalı işle dolu, diskler beklemiyor.
- Düşük ek yük: işletim sisteminin kendisi az kaynak kullanmalıdır.

3. Gelişebilirlik

- Yeni donanımlar (SSD, GPU, 5G modem) ve yeni hizmetler eklenebilmeli…
- …mevcut programları bozmadan.
- İşletim sistemi yapısının seçimini belirler (4. bölüm).

Daha sonra ele alacağımız her yapı, bu üç hedef arasında farklı bir uzlaşmadır.

Hangi Hedef Daha Önemli? Makineye Göre Değişir

Sistem türü	Ana öncelik	Neden
Kişisel bilgisayar	Kolaylık	"Sorunsuz çalışmasını" isteyen tek bir kullanıcı; CPU zaten çoğu zaman boştadır.
Sunucu / bulut	Verimlilik	Binlerce kullanıcı; %1 daha fazla iş hacmi, yüzlerce makine daha az demek olabilir.
Akıllı telefon	Verimlilik (enerji)	Pil ömrü: işletim sistemi uygulamaları ve CPU'yu agresif biçimde uyku moduna alır.
Gömülü / IoT	Verimlilik (boyut, maliyet)	Bir çamaşır makinesi denetleyicisinin yalnızca birkaç kilobayt belleği olabilir; hiç işletim sistemi olmayabilir.
Gerçek zamanlı (araç frenleri, dronlar)	Öngörülebilirlik	Geç gelen yanıt yanlış yanıttır: süre sınırlarına her zaman uyulmalıdır.

Kolaylık ve Verimlilik: Çözümlü Bir Örnek

Durum

- Bir program bir dosyadan 1.000.000 sayı okumalıdır.
- **Kolay yol:** işletim sisteminden her seferinde bir sayı istemek → 1.000.000 sistem çağrısı.
- **Verimli yol:** 4.096 baytlık bloklar istemek (her biri ≈ 1.000 sayı) → yaklaşık 1.000 sistem çağrısı.
- Bir sistem çağrısının salt ek yükü yaklaşık 0,1 µs ise (bunu bugün ölçeceğiz):

```
tek tek:
  1.000.000 × 0,1 µs = 100 ms

1.000'lik bloklar hâlinde:
  1.000 × 0,1 µs = 0,1 ms

=> 1000 kat daha az ek yük
```

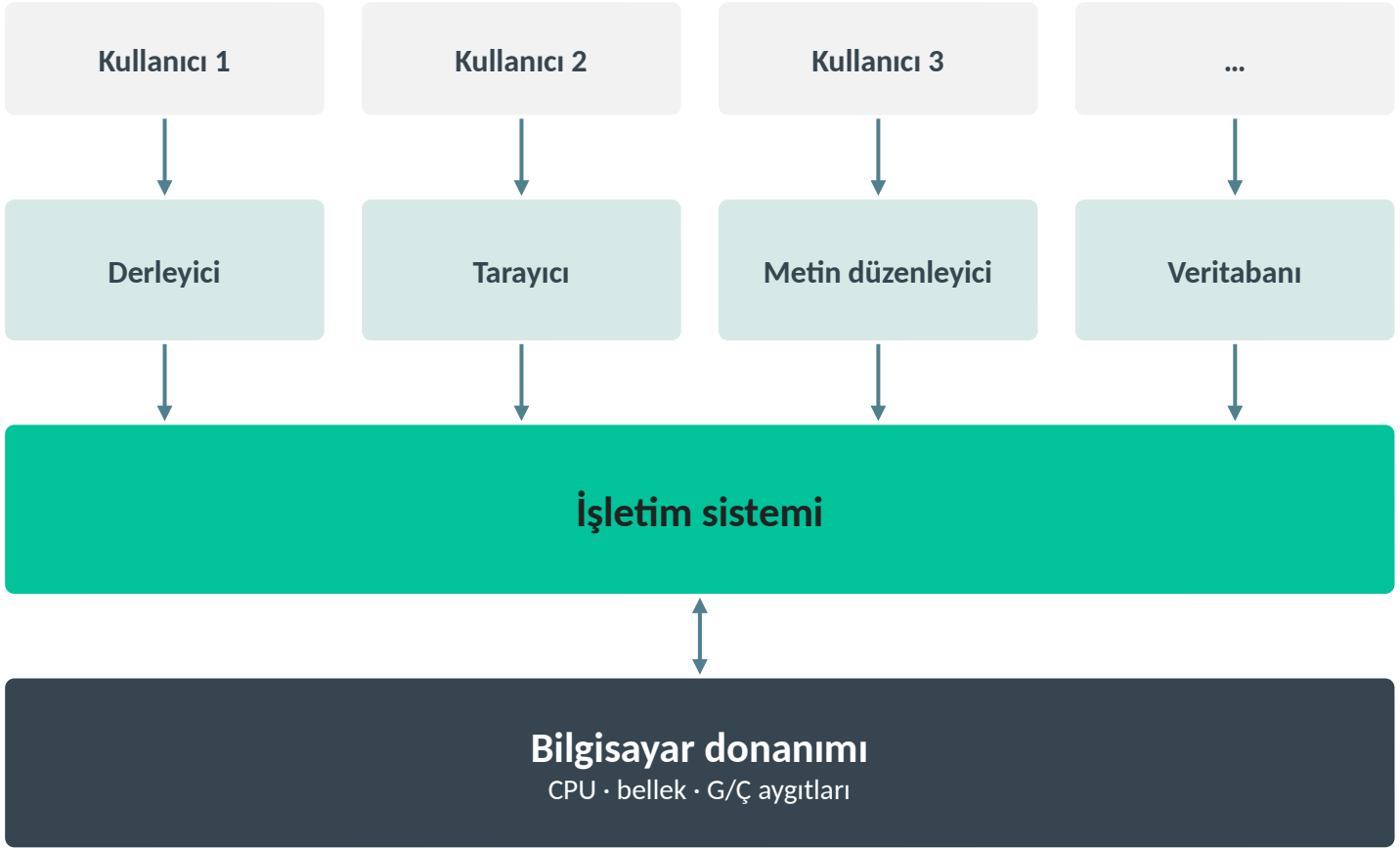
C'nin stdio gibi kütüphaneleri ikisini birden sunar: fgetc()'yi her seferinde tek bir karakter için çağırırsınız, ama kütüphane arka planda büyük bloklar okur.

3. BÖLÜM

Bir Bilgisayar Sisteminin Bileşenleri

Donanım, işletim sistemi, uygulama programları ve kullanıcılar.

Bir Bilgisayar Sisteminin Dört Bileşeni

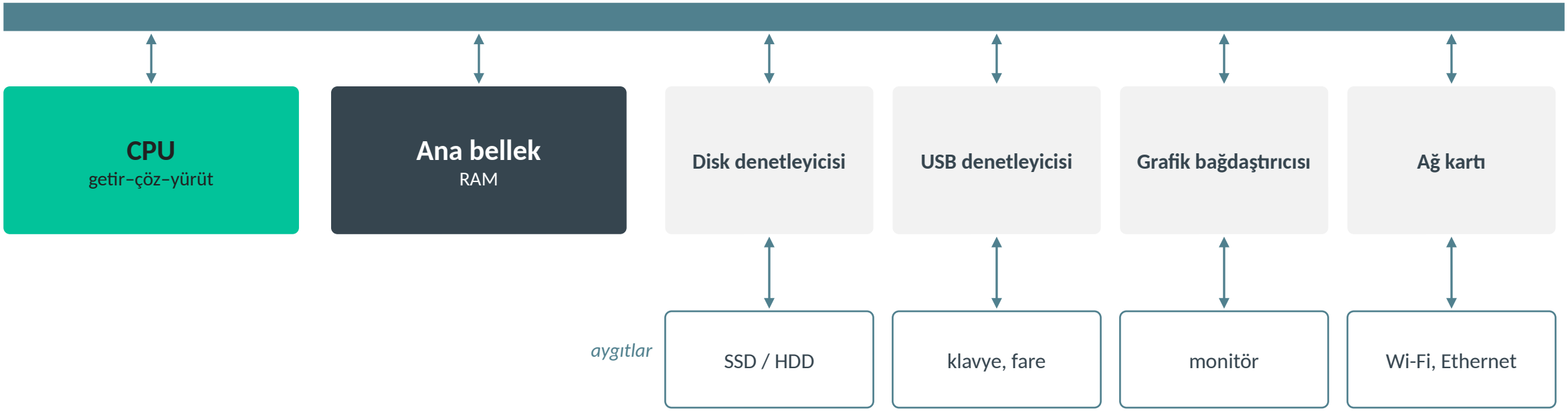


Roller

- **Donanım** temel hesaplama kaynaklarını sağlar.
- **Uygulama programları** bu kaynakları kullanıcıların sorunlarını çözmek için kullanır.
- **Kullanıcılar:** insanlar, ama aynı zamanda başka makineler ve programlar.
- **İşletim sistemi** donanımı denetler ve onun uygulamalar ile kullanıcılar arasındaki kullanımını koordine eder.

Kutunun İçi: Sistem Yolu, CPU, Bellek ve Aygıtlar

sistem yolu (bus): veri, adres ve denetim sinyallerini taşır



Her aygıt denetleyicisinin küçük bir yerel tamponu vardır ve CPU ile paralel çalışır. İşletim sistemi her denetleyici için bir aygıt sürücüsü içerir: sürücü denetleyicinin dilini konuşur, böylece işletim sisteminin geri kalanının bu dili bilmesi gerekmez.

Kesmeler: Aygıt CPU'nun Dikkatini Nasıl Çeker?

CPU klavyeye sürekli "yeni bir şey var mı?" diye sormaz; klavye CPU'ya haber verir.

1. Aygıt bitirir

Disk denetleyicisi veriyi kendi tamponuna kopyalamıştır.

→

2. Sinyal

CPU üzerindeki bir kesme hattını etkinleştirir.

→

3. Kaydet, atla

CPU o anki işini kaydeder ve kesme vektöründe yazılı işleyiciye atlar.

→

4. İşle

İşletim sisteminin kesme işleyicisi (sürücünün parçası) olayı işler.

→

5. Devam et

Kaydedilen durum geri yüklenir; kesilen program hiçbir şey olmamış gibi devam eder.

Modern işletim sistemleri kesme güdümlüdür: çalışan program yoksa ve hiçbir aygıt ilgi beklemiyorsa işletim sistemi yalnızca bekler. Her tuş basımı, her ağ paketi ve biten her disk okuması bir kesme olarak gelir; saniyede binlercesi.

Depolama Hiyerarşisi: Hızlı ve Küçük, Yavaş ve Büyük

Düzey	Tipik boyut	Tipik erişim süresi	Bir yazmaç erişimi 1 saniye sürseydi…
CPU yazmaçları	< 1 KB	≈ 0,3 ns	1 saniye
Önbellek (L1–L3)	birkaç MB	1 – 40 ns	3 saniye – 2 dakika
Ana bellek (RAM)	8 – 64 GB	≈ 100 ns	≈ 5 dakika
SSD	0,5 – 4 TB	≈ 50 µs	≈ 2 gün
Sabit disk (HDD)	1 – 20 TB	≈ 5 ms	≈ 6 ay

Yazmaçlar, önbellek ve RAM uçucudur (güç kesilince içerik kaybolur); SSD'ler ve diskler kalıcıdır. İşletim sistemi verileri bu düzeyler arasında sürekli taşır: 6. ve 7. haftaların konusu.

Çift Modlu Çalışma: Kullanıcı Modu ve Çekirdek Modu

Kimin neyi yapabileceğini donanımın kendisi, CPU içindeki bir mod biti ile denetler.



Neden önemli?

- **Ayrıcalıklı komutlar** (bir aygıtla konuşmak, bellek korumasını değiştirmek, CPU'yu durdurmak) yalnızca çekirdek modunda çalışır.
- Bir kullanıcı programı böyle bir komutu denerse donanım işletim sistemine bir tuzak (trap) üretir; işletim sistemi genellikle programı sonlandırır.
- Böylece hatalı ya da kötü niyetli bir uygulama makineyi ele geçiremez.
- x86 işlemcilerde modlara halka (ring) denir: Linux ve Windows halka 0'ı (çekirdek) ve halka 3'ü (kullanıcı) kullanır.

UYGULAMA

1. Örnek: Çekirdeği Çağıran Bir Program

```c
/* hello_syscall.c -- aynı mesajı iki farklı yolla yazdırmak.
 * Derleme   : gcc hello_syscall.c -o hello_syscall
 * Çalıştırma: ./hello_syscall
 * İzleme    : strace ./hello_syscall   (yalnızca Linux)
 */
#include <stdio.h>   /* printf: kütüphane fonksiyonu */
#include <unistd.h>  /* write: sistem çağrısı sarmalayıcısı */

int main(void)
{
    /* 1) Kütüphane çağrısı: printf metni kullanıcı modunda
     *    biçimlendirir, sonra write() ile çekirdeğe yazdırır. */
    printf("printf ile merhaba!\n");
    fflush(stdout);

    /* 2) Doğrudan sistem çağrısı: 1 numaralı dosya tanımlayıcısı
     *    = ekran (standart çıktı); 21 = yazılacak bayt sayısı. */
    write(1, "write() ile merhaba!\n", 21);

    return 0;
}
```

Ne oluyor?

- printf() bir kütüphane fonksiyonudur: metni kullanıcı modunda biçimlendirir.
- write() bir sistem çağrısıdır: çekirdek moduna geçilir ve baytları ekrana çekirdek gönderir.
- strace, bir programın yaptığı her sistem çağrısını gösterir:

```
$ strace ./hello_syscall
write(1, "printf ile merhaba!\n", 20)   = 20
write(1, "write() ile merhaba!\n", 21)  = 21
exit_group(0)                           = ?
```

2. Örnek: Çekirdeğe Gidip Gelmek Ne Kadar Pahalı?

```c
#define N 1000000L

long plain_function(long x) { return x + 1; }

int main(void)
{
    volatile long sink = 0;
    double t0 = seconds();
    for (long i = 0; i < N; i++)
        sink += plain_function(i);    /* yalnız kullanıcı modu */
    double t1 = seconds();
    for (long i = 0; i < N; i++)
        sink += syscall(SYS_getpid);  /* çekirdeğe gidip döner */
    double t2 = seconds();

    printf("sıradan fonksiyon çağrısı : %6.1f ns\n",
           (t1 - t0) / N * 1e9);
    printf("sistem çağrısı (getpid)   : %6.1f ns\n",
           (t2 - t1) / N * 1e9);
}
/* programın tamamı: code/syscall_cost.c */
```

Deney

- Sıradan bir fonksiyonu bir milyon kez çağırın.
- Ardından bir milyon küçük sistem çağrısı (getpid) yapın.
- Çağrı başına ortalama süreyi karşılaştırın.
- Kesin sayılar makinenize bağlıdır; asıl ders orandadır.

```
$ ./syscall_cost
sıradan fonksiyon çağrısı :    1.5 ns
sistem çağrısı (getpid)   :  116.1 ns
sistem çağrısı yaklaşık 76 kat daha yavaş
```

ARA

10 Dakika Ara

Sırada: işletim sistemleri nasıl inşa edilir? MS-DOS'tan mikroçekirdeklere.

4. BÖLÜM

İşletim Sistemi Yapıları

Aynı hedefler, çok farklı binalar.

Yapı Neden Önemlidir?

Linux çekirdeği 30 milyondan fazla kod satırından oluşur. Bu kadar büyük bir şeyi nasıl düzenlersiniz?

Basit

Neredeyse hiç yapı yok (MS-DOS).

Monolitik

Tek büyük bir çekirdek programı (UNIX, Linux).

Katmanlı

Katı katmanlar; her biri yalnızca altındakini kullanır (THE).

Mikroçekirdek

Küçücük çekirdek; hizmetler kullanıcı programıdır (Mach, MINIX 3, QNX).

Modüler

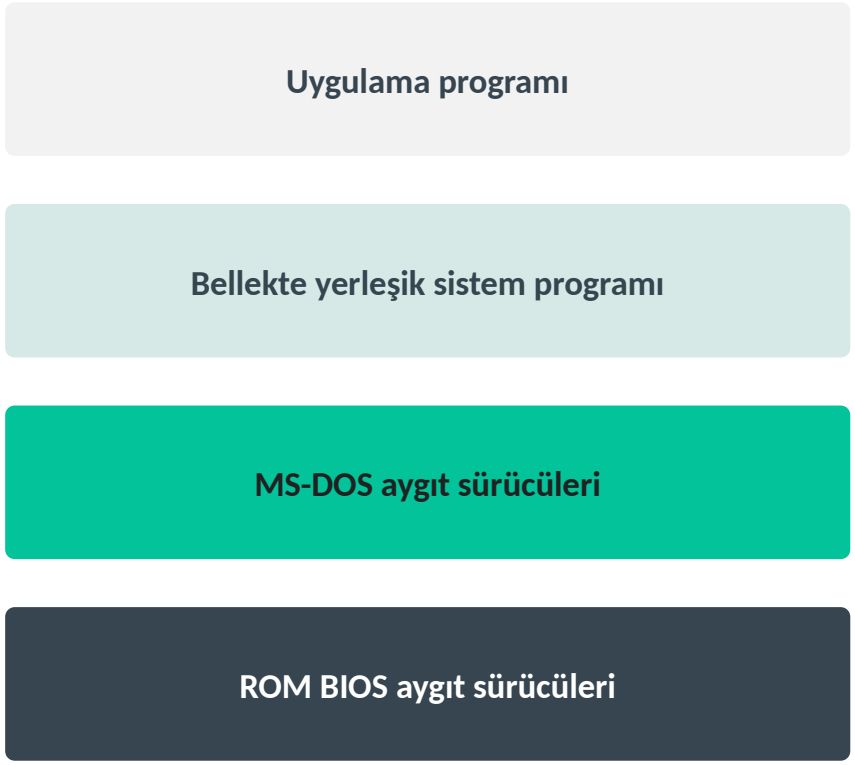
Ana çekirdek + yüklenebilir modüller (Linux).

Hibrit

Gerçek sistemler bunları karıştırır (Windows, macOS, Android).

Basit Yapı: MS-DOS

1981'deki IBM PC'nin çok küçük belleğine sığmak için yazıldı; hiçbir zaman gerçek anlamda katmanlı tasarlanmadı.



Uygulamalar katmanları atlayıp donanıma doğrudan erişebiliyordu.

Avantajlar

- Çok küçük ve hızlıydı.
- Sınırlı donanım için yazması kolaydı.

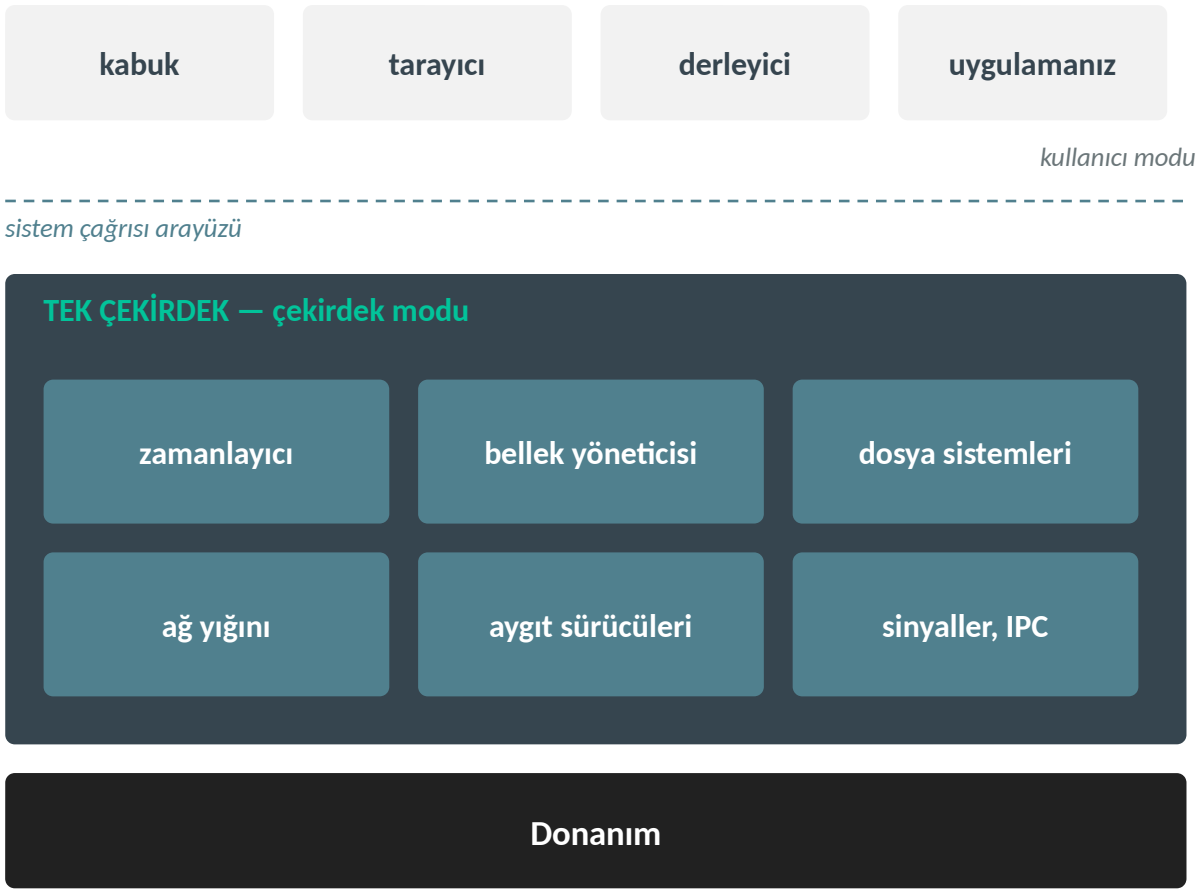
Dezavantajlar

- Koruma yok: tek hatalı program tüm sistemi çökertir.
- 8088 işlemcide çift mod yoktu; İS bunu önleyemezdi.

Ders: koruma, donanım desteği (çift mod) ve onu kullanan bir işletim sistemi gerektirir.

Monolitik Çekirdek: UNIX ve Linux

Tüm işletim sistemi hizmetleri, çekirdek modunda ve tek bir adres alanında çalışan büyük bir programda bulunur.



Avantajlar

- Hızlı: hizmetler birbirini doğrudan çağırır.
- Sistem çağrısı başına yalnızca iki mod geçişi.

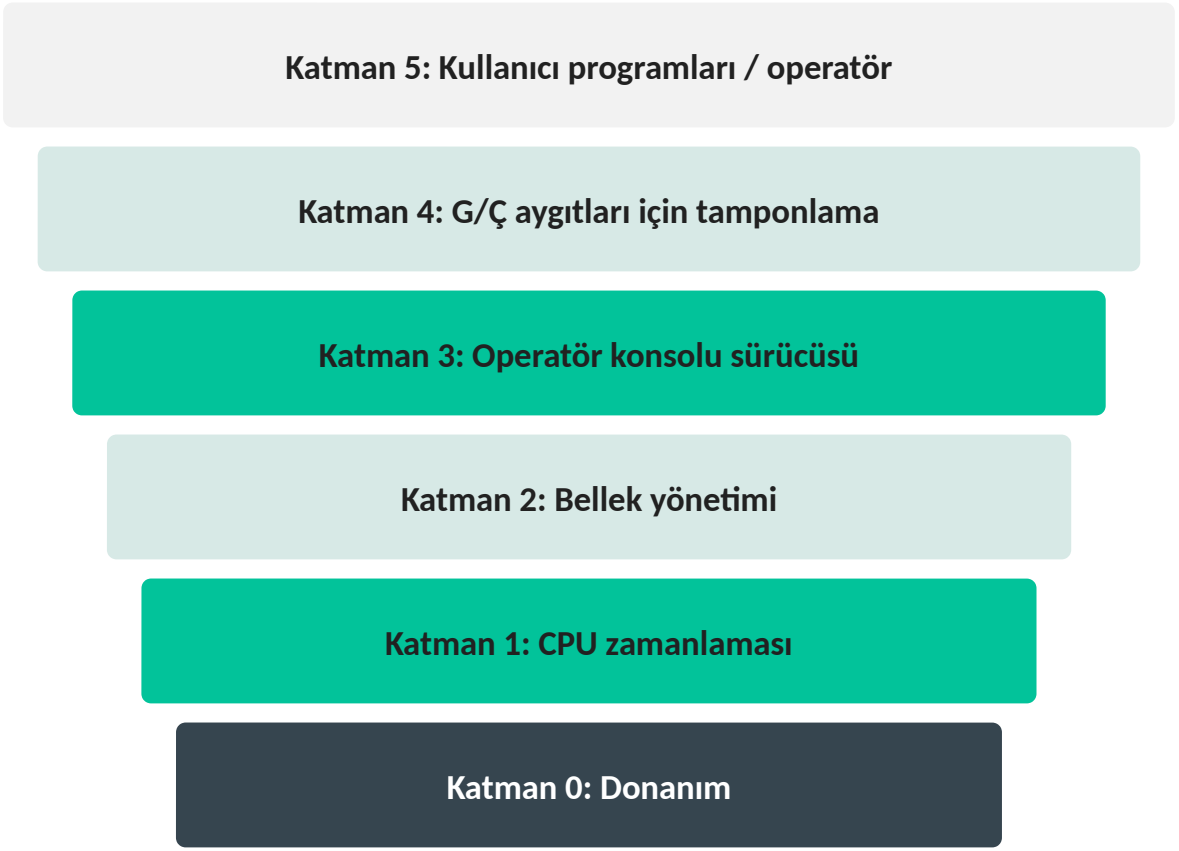
Dezavantajlar

- Çok büyük ve bakımı zor.
- Tek bir sürücü hatası tüm çekirdeği çökertebilir ("kernel panic").

Örnekler: ilk UNIX (1970'ler), Linux, geleneksel BSD.

Katmanlı Yaklaşım

İşletim sistemi katmanlara bölünür; N. katman yalnızca altındaki katmanları kullanabilir. 0. katman donanımdır.



Dijkstra'nın THE sistemine (1968) gevşek biçimde dayanır.

Avantajlar

- Katman katman inşa etmek ve hata ayıklamak kolaydır.
- Her katman ayrı ayrı doğrulanabilir.

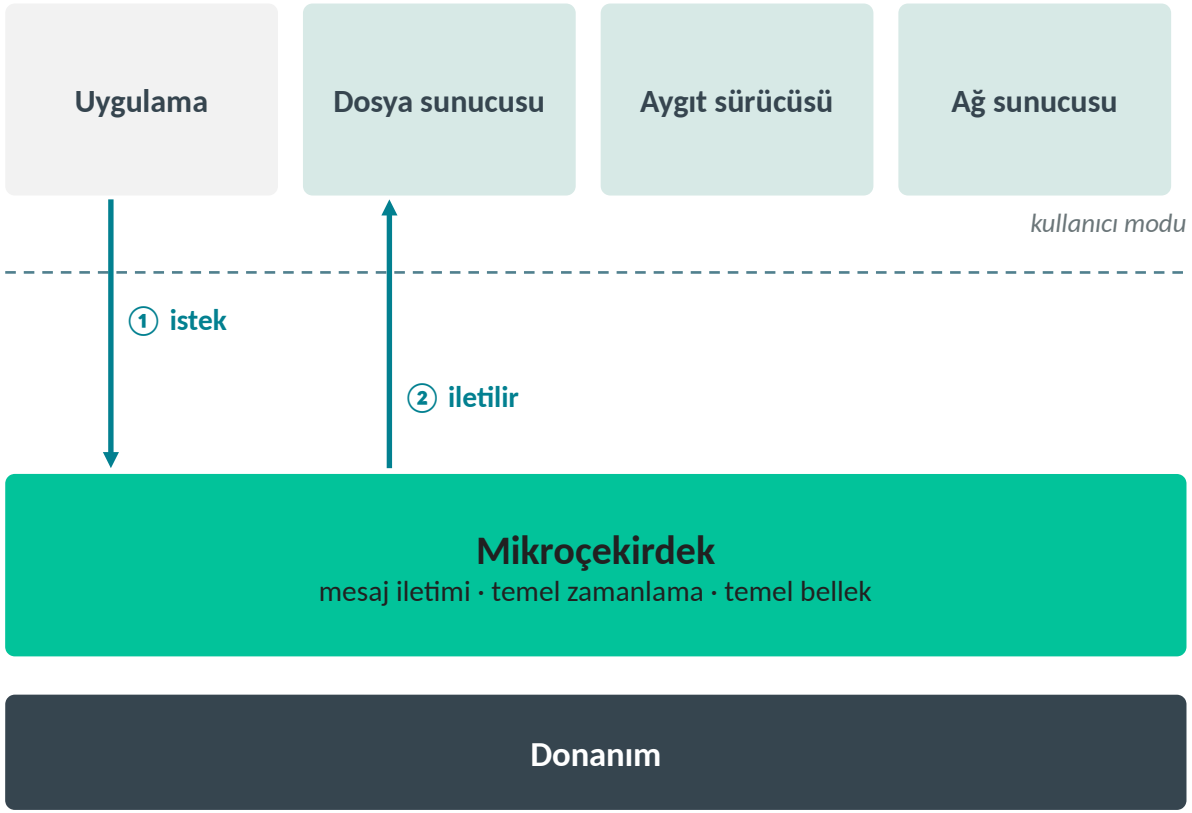
Dezavantajlar

- Sırayı belirlemek zordur (bellek zamanlayıcıya mı dayanır, tersi mi?).
- Her istek birçok katmandan geçer → yavaş.

Saf katmanlı yapı bugün nadirdir, ama fikir her işletim sisteminin içinde yaşar (örneğin ağ yığını katmanlıdır).

Mikroçekirdek: Çekirdeği Küçük Tut

Çekirdek modunda yalnızca en temel kısım kalır; geri kalan her şey kullanıcı modundaki sıradan sunucular olarak çalışır.



Uygulama dosya sunucusunu doğrudan çağırmaz: mesajlar çekirdek üzerinden gider.

Avantajlar

- Güvenilir: çöken sürücü yeniden başlatılır.
- Güvenli, genişletilebilir; küçük çekirdek kolay doğrulanır.

Dezavantajlar

- Daha yavaş: her hizmet isteği ek mesajlar ve mod geçişleri gerektirir.

Örnekler: Mach (1980'ler), MINIX 3, QNX (otomobiller, tıbbi cihazlar), seL4 (matematiksel olarak doğrulanmış).

Çözümlü Örnek: Mikroçekirdeğin Bedeli

Bir dosya okuması için mod geçişlerini sayalım

- **Monolitik:** uygulama → çekirdek → uygulama. 2 geçiş.
- **Mikroçekirdek:** uygulama → çekirdek → dosya sunucusu → çekirdek → uygulama. 4 geçiş, artı 2 mesaj kopyası.
- Varsayalım (temsilî sayılar): bir geçiş = 0,1 µs, bir mesaj kopyası = 0,15 µs, faydalı iş = 1 µs.

```
Monolitik:
  T = 2 × 0,1 + 1,0 = 1,2 µs

Mikroçekirdek:
  T = 4 × 0,1 + 2 × 0,15 + 1,0 = 1,7 µs

Yavaşlama: 1,7 / 1,2 ≈ 1,42 (%42)
```

Kısa ve sık işlemlerde ek yük baskın çıkar. Uzun işlemlerde (büyük bir dosyayı okumak) neredeyse önemsizdir. L4 gibi modern mikroçekirdekler mesajları ucuzlatmak için çok çalışır.

Modüler Çekirdek: Yüklenebilir Çekirdek Modülleri

Bir ana çekirdek ve sistem çalışırken eklenip çıkarılabilen modüller.



Modüller gerektiğinde yüklenir ve çalışan çekirdeğe bağlanır.

Avantajlar

- Monolitik gibi hızlı: modüller çekirdek modunda çalışır.
- Mikroçekirdek gibi esnek: sürücü gerektiğinde yüklenir.

Dezavantajlar

- Hatalı bir modül de çekirdek modunda çalışır ve sistemi çökertebilir.

Örnekler: Linux (lsmod, insmod, rmmod), Solaris, modern BSD'ler. Bugün en yaygın tasarım.

UYGULAMA

3. Örnek: C ile Yüklenebilir Bir Çekirdek Modülü

```c
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("MBLP131");
MODULE_DESCRIPTION("Merhaba dünya çekirdek modülü");

static int __init hello_init(void)    /* insmod sırasında çalışır */
{
    pr_info("hello_module: loaded into the kernel\n");
    return 0;                          /* 0 = başarılı */
}

static void __exit hello_exit(void)   /* rmmod sırasında çalışır */
{
    pr_info("hello_module: removed from the kernel\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

Nasıl kullanılır (Linux)

- Çekirdeğin kendi derleme sistemiyle derleyin (code klasöründeki Makefile).
- pr_info() ekrana değil, çekirdek günlüğüne yazar.
- Yalnızca bir sanal makinede deneyin.

```
$ make
$ sudo insmod hello_module.ko
$ lsmod | grep hello
$ sudo dmesg | tail -1
hello_module: loaded into the kernel
$ sudo rmmod hello_module
```

Hibrit Sistemler: Gerçek Sistemler Ne Yapar?

Saf tasarımlar nadirdir. Gerçek sistemler başarım, güvenlik ve kolaylık için birkaç yapıdan ödünç alır.

Windows (NT ailesi)

- Mikroçekirdek fikirleriyle tasarlandı (ayrı alt sistemler).
- Hız için grafik dâhil çoğu hizmet çekirdek modunda çalışır.
- Modüller gibi yüklenebilir sürücüler.

macOS ve iOS

- Çekirdek "XNU" = Mach mikroçekirdeği + BSD UNIX parçaları, tek bir çekirdek olarak derlenir.
- Sürücüler yüklenebilir çekirdek uzantılarıdır.

Android

- Değiştirilmiş bir Linux çekirdeği (monolitik + modüller).
- Üstünde: donanım soyutlama katmanı, yerel kütüphaneler ve uygulamalar için Android çalışma ortamı.

1992: Andrew Tanenbaum (MINIX, mikroçekirdek), monolitik olduğu için Linus Torvalds'a "Linux artık eskidi" dedi. Otuz yılı aşkın süre sonra iki tasarım da hâlâ her gün kullanılıyor.

ÖZET

Yapılar Yan Yana

Yapı	Çekirdek modunda	Hız	Güvenilirlik	Örnek
Basit	her şey (koruma yok)	yüksek	çok düşük	MS-DOS
Monolitik	tüm İS hizmetleri	yüksek	orta	UNIX, Linux
Katmanlı	tüm katmanlar	düşük	yüksek	THE (1968)
Mikroçekirdek	yalnızca küçük çekirdek	daha düşük	yüksek	MINIX 3, QNX, seL4
Modüler	çekirdek + yüklenen modüller	yüksek	orta	Linux, Solaris
Hibrit	değişir	yüksek	orta–yüksek	Windows, macOS, Android

Yinelenen ödünleşim: eklediğiniz her sınır (katman, mesaj, mod geçişi) güvenilirlik kazandırır, hızdan götürür.

KENDİNİZ DENEYİN

Kendi İşletim Sisteminizin İçine Bakın

```
# Linux (laboratuvar PC'leri, WSL ya da sanal makine)
$ uname -r          # çekirdek sürümü nedir?
$ lsmod | head      # hangi modüller yüklü?
$ strace -c ls      # ls'nin sistem çağrılarını say

# Windows
> winver            # hangi Windows sürümü?
> driverquery       # yüklü sürücüleri listele
# Görev Yöneticisi > Performans: CPU süresi
# "kullanıcı" ve "çekirdek" süresine ayrılır
```

Yanıtlanacak sorular

- Makinenizde kaç çekirdek modülü yüklü?
- ls en sık hangi sistem çağrısını kullanıyor?
- ls gibi basit bir komut kabaca kaç sistem çağrısı yapar? (Test makinemizde: 76.)
- Görev Yöneticisi'ni açın ve çekirdek modunda çok zaman geçiren bir program bulun. Bunun nedeni ne olabilir?

KAPANIŞ

2. Hafta — Temel Çıkarımlar

- **Tanım:** işletim sistemi donanımı yönetir ve programlara daha basit, daha güvenli bir makine sunar; bellekte sürekli bulunan özü çekirdektir.
- **İki bakış:** genişletilmiş makine (soyutlama, yukarıdan aşağıya) ve kaynak yöneticisi (çoğullama, aşağıdan yukarıya).
- **Üç hedef:** kolaylık, verimlilik, gelişebilirlik; PC'lerde, sunucularda, telefonlarda ve gömülü sistemlerde farklı ağırlıklarla.
- **Dört bileşen:** donanım, işletim sistemi, uygulama programları, kullanıcılar.
- **Koruma donanım ister:** çift mod (kullanıcı/çekirdek) ve kesmeler; bir sistem çağrısı, bir fonksiyon çağrısından çok daha pahalıdır.
- **Yapılar:** basit, monolitik, katmanlı, mikroçekirdek, modüler, hibrit; her biri hız ile güvenilirlik arasında farklı bir uzlaşma.

KISA SINAV

Anladıklarınızı Kontrol Edin

1. İşletim sisteminin yaptığı zamanda çoğullamaya ve alanda çoğullamaya birer örnek verin.

2. MS-DOS'taki bir programın hatası neden tüm bilgisayarı çökertebilirdi de Linux'taki bir programınki çökertemez?

3. Bir sistem çağrısı 100 ns, bir fonksiyon çağrısı 1,5 ns sürüyor. 10^6 sistem çağrısı, 10^6 fonksiyon çağrısına göre ne kadar ek yük getirir?

4. Mikroçekirdeğin monolitik çekirdeğe göre bir avantajını ve bir dezavantajını söyleyin.

GELECEK HAFTA

Hizmetler, Sistem Çağrıları ve Süreçler

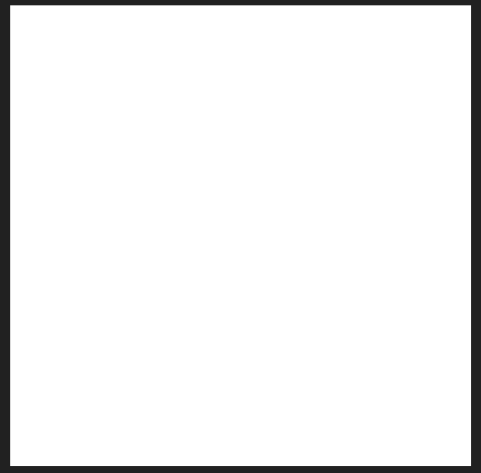
3. haftanın konuları:

- İşletim sisteminin sunduğu hizmetler
- Ayrıntılı olarak sistem çağrıları: türleri, parametreleri, sistem çağrısı tablosu
- Süreç kavramı: yürütülmekte olan bir program
- Süreç durumları: yeni, hazır, çalışıyor, bekliyor, sonlandı

SORULAR?

Tartışma

Dersten sonra sorularınız olursa dersin Telegram kanalında sorun — yanıttan herkes yararlanır.



Dr. Öğr. Üyesi Hasan Oğuz

Ofis:
[ders grubu bağlantısı]